

BreastCancerDetectionCNN

Senior Design Project Plan — Fall 2026, Milestones 4–6

Team

Members

Kahlel Cardona kcardona2023@my.fit.edu

Woroma Dimkpa wdimkpa2021@my.fit.edu

Taratong Dolinsky tdolinsky2022@my.fit.edu

Faculty Advisor Dr. Zahra Nematzadeh znematzadeh@fit.edu

Date of Meeting with the Client for Developing this Plan: 08/27/26

Goal and Motivation

The overall goal of the project is to develop a machine learning-based system that assists medical professionals and researchers by automatically classifying breast tumors as benign or malignant using mammogram images from the CBIS-DDSM dataset. By improving accuracy, consistency, and efficiency in tumor classification, the system aims to support earlier detection and reduce diagnostic burden, ultimately improving clinicians' and patients' confidence in screening outcomes. Breast Cancer is one of the most prevalent cancers worldwide and remains a leading cause of cancer-related deaths among women. Early detection is critical, as it significantly increases the chances of successful treatment and long-term survival.

Mammography is one of the most widely used screening methods; however, interpreting mammogram images is a challenging and time consuming task that requires significant expertise. Radiologists and clinicians often face issues such as image noise, subtle tumor patterns, and high workloads, which can lead to delayed diagnosis or misclassification of tumors. Current systems and manual analysis methods are limited by human subjectivity, fatigue, and variability in experience. Additionally, traditional computer-aided diagnosis systems may rely on handcrafted features that fail to generalize well across diverse imaging data. Advances in deep learning, particularly Convolutional Neural Networks (CNNs), provide an opportunity to overcome these limitations by learning discriminative features directly from medical images. This project is motivated by the need for more reliable, scalable, and data-driven diagnostic support tools in healthcare.

Approach (Key Features of the System)

Feature 1 — Dual-model mammogram classification through a simple web upload

The user can open a web page in any browser, choose a mammogram image file from their own computer (.jpg, .jpeg, or .png), and click a single button to get an answer back within a few

seconds, with no installation, command line, or account required. Behind that one click, the system runs the uploaded image through two separately trained neural networks a ResNet-50-based classifier and an EfficientNet-B0-based classifier and the user sees both models' individual verdicts ("Benign" or "Malignant") side by side, each with its own confidence percentage, so the user is never given a single answer without also being shown how confident, and how independently corroborated, that answer is. If the user submits no file, or a file that is not a readable image, the page tells them clearly what went wrong instead of crashing or hanging.

Feature 2 — Confidence-weighted "Final Answer" with explicit disagreement flagging

The user can see, beneath the two individual model verdicts, one blended "Final Answer" with its own confidence percentage, produced by combining the two models' probability-of-malignancy scores weighted by each model's own measured accuracy on real, unseen test data not by an assumed or self-reported number. Critically, when the two underlying models disagree with each other (which does happen in practice; one verified example in this project's own testing had ResNet say "Malignant" at 88% confidence while EfficientNet said "Benign" at 65% confidence, correctly resolving to a 62%-confidence "Malignant" final answer), the user is shown an explicit on-page note that the models disagreed, rather than the disagreement being silently hidden inside an averaged number. This lets a user calibrate how much to trust a given result: strong agreement between two independently-trained architectures is a meaningfully different situation than one model narrowly outvoting the other, and only this feature makes that distinction visible.

Feature 3 — A rigorous, honest model-evaluation and comparison workflow

The user in this feature's case, primarily the development team and any researcher inspecting this project's claims can run a single command-line tool (`evaluate.py`) against any saved model checkpoint (or a list of checkpoints, for ensembling) and get back accuracy, precision, recall, and F1 score measured strictly on a held-out set of mammograms that were never used in training or validation for that model, with test-time augmentation and automatic per-model decision-threshold tuning built in. The user can also open a separate desktop dashboard (`CompareModels.py`) to browse the training-curve plots and per-fold metrics produced by each architecture's 5-fold cross-validation run, letting them visually compare how the two architectures trained over time rather than only seeing a single final number. This is the feature that already caught and corrected this project's own early overstated accuracy figure, and it is treated as a first-class feature of the system, not an internal testing detail, precisely because trustworthy self-reporting is central to this project's goal.

Algorithms and Tools

The system is built entirely in Python 3.13 on top of PyTorch and torchvision. Both classifiers use transfer learning from ImageNet-pretrained backbones torchvision's resnet50 (wrapped in a custom `ResNetWithAttnPool` class that adds a learned attention-pooling layer before the classification head) and torchvision's efficientnet_b0 with early backbone layers frozen at the start of training and unfrozen in stages on a fixed epoch schedule, so the network adapts

progressively rather than all at once. Training uses a hybrid loss that blends standard cross-entropy (with label smoothing) and focal loss (which down-weights easy, already-correct examples so the model focuses on the harder, often minority-class malignant cases), implemented in a shared losses.py module used by both architectures. Both models use mixup (blending pairs of training images and labels), gradient-norm clipping, and 5-fold cross-validation via scikit-learn's StratifiedKFold. Model quality is measured with scikit-learn's accuracy_score, precision_score, recall_score, and f1_score, and every evaluation additionally supports three-way test-time augmentation (original image, horizontal flip, vertical flip, logits averaged) to reduce prediction variance.

The dataset is the CBIS-DDSM (Curated Breast Imaging Subset of the Digital Database for Screening Mammography), a public, de-identified research dataset of 10,237 mammogram JPEG images and five CSV metadata files describing mass and calcification findings with ground-truth benign/malignant pathology labels, loaded through a custom PyTorch Dataset class that also applies pixel-statistic analysis (counting distinct grayscale values) to reliably tell a real diagnostic image crop apart from a same-folder region-of-interest mask, since the two cannot be told apart by filename alone. The web application is a Flask app serving a single upload-and-result page (HTML/CSS, no JavaScript framework), and the desktop comparison dashboard is built with Python's built-in Tkinter. Training metrics are logged to Excel via openpyxl.

Novel Features

- Confidence-weighted dual-architecture ensembling with explicit disagreement detection: rather than picking one “best” model and discarding the other, the system keeps both architectures' opinions visible and mathematically blends them by each model's own measured (not assumed) reliability, and it is novel in this context because it treats model disagreement as user-facing information rather than an implementation detail to be averaged away silently.
- Group-aware cross-validation (StratifiedGroupKFold) that keeps a lesion's full image and its cropped patch in the same fold, preventing a subtle form of validation-score inflation (a model being tested on an image whose close relative it already trained on) that this project identified as a genuine risk before it could quietly distort reported results.

Technical Challenges

Challenge 1 Closing the accuracy gap without a clear, proven lever. On the honest held-out test set, both architectures currently sit in the mid-60s percent accuracy range (F1 around 0.63–0.65), and three different, carefully isolated improvement attempts this year adding roughly 2,800 real cropped-patch training images, per-model decision-threshold tuning combined with 5-fold checkpoint ensembling, and strengthened synthetic data augmentation (currently mid-retrain as of this plan) have each produced results that are flat to only marginally positive, and in one case (EfficientNet with added crop images) measurably worse. We do not yet know which lever, if any, will move held-out performance meaningfully: candidates for this semester include trying a different or larger backbone architecture, exploring knowledge distillation (a teacher-student approach briefly attempted in an earlier, now-abandoned checkpoints_distill/

effort with no working script left in the repository), or sourcing additional external mammography data beyond CBIS-DDSM.

Challenge 2 A persistent validation-versus-test “optimism gap.” Every retraining experiment this project has run shows validation F1 in the 0.70–0.75 range but held-out test F1 in the 0.55–0.65 range a large, consistent gap that has not closed despite three different intervention attempts. The leading hypothesis, not yet directly tested, is that the validation folds (drawn from the same 2,864–5,634 training-derived images used to fit the model) share enough close visual relationship with their own training images that validation performance partly reflects memorization rather than generalization, while the truly held-out test images come from different underlying patient studies entirely. Determining whether this is the real cause, versus dataset noise or a genuine distribution shift between the train and test splits as originally partitioned by CBIS-DDSM, is an open technical problem this semester's work must make real progress on, since it currently limits how much confidence can be placed in any single validation-based decision.

Components / Modules

Module	Responsibility
DataSetAugmentation.py	Dataset loading (MammogramRawDataset), crop/mask disambiguation, train/val transform pipelines
losses.py	Shared FocalLoss, mixup_data, and hybrid focal+cross-entropy criterion builder
training_log.py	Per-epoch / per-fold Excel logging (openpyxl), saved incrementally during training
ResNet.py	ResNet-50 + attention-pooling model, SAM optimizer, 5-fold training loop
EfficientNet.py	EfficientNet-B0 model, staged unfreezing, cosine-restart LR, 5-fold training loop
evaluate.py	Held-out test evaluation, TTA, per-model threshold search, multi-checkpoint ensembling
inference.py	Loads selected checkpoints; runs dual-model prediction and confidence-weighted blending
app.py / templates / static	Flask web app: upload form, result rendering, styling
CompareModels.py	Tkinter desktop dashboard for browsing training-curve plots across architectures

Interfacing with Users

Two distinct user types are served by two distinct interfaces. End users (clinicians, radiology researchers, or biomedical-imaging students evaluating the tool) interact only through the Flask web application's single upload page no code or command line is ever required. Developers and researchers extending or auditing the project interact through the command line (evaluate.py, the training scripts) and through the Tkinter-based CompareModels.py dashboard for visual inspection of training curves and per-fold metrics; this project also maintains checked-in, dated

ExecPlan documents (specs/*.md) as a written interface for any future team member to understand exactly what was tried and why.

Evaluation

Success this semester will be measured along four dimensions:

- Accuracy / F1: the held-out test-set F1 score (evaluate.py, 704-image untouched test set, 428 benign / 276 malignant) for each architecture, compared explicitly against this project's own established baseline (ResNet F1 0.6534, EfficientNet F1 0.6375) — the semester's technical-challenge work is considered successful only if it produces a real, honestly-measured improvement over this baseline, not merely a different number.
- Speed: end-to-end latency from image upload to a rendered result on the web app, measured on the team's GPU workstation; target is under 2 seconds per image including both models' forward passes and TTA.
- Reliability: repeated-trial success rate of the web app across a batch of real sample images and known edge cases (no file submitted, non-image file submitted), following the same pass/fail methodology already used to accept this project's prior milestones (target: 10/10 successful, non-crashing responses).
- Reproducibility: whether a full retraining run produces internally consistent results (checkpoints of identical size/shape across all 5 folds, no load errors, and an Excel log with the expected row count) — already used as a concrete, automatable acceptance bar for every training change in this project.
- User feedback (once the Faculty Advisor is assigned and available): a 1–5 rating of the web app's clarity, trustworthiness of the confidence display, and usefulness of the disagreement note, gathered from the advisor and, where possible, from CSE peers or clinical contacts reviewing the live application.

Progress Summary

Module / Feature	Completion %	To Do
Dataset loading & augmentation (DataSetAugmentation.py)	100%	Revisit only if a future data-source change requires it
ResNet-50 training pipeline	90%	Resolve validation/test optimism gap; evaluate next architectural lever
EfficientNet-B0 training pipeline	90%	Same as above
Inference module	50%	Compile it
Flask web app	10%	Create upload, login, ui, database
CompareModels.py desktop dashboard	85%	Remove/repair stale checkpoints_distill and plots_distill_kfold references
Training/eval Excel logging (training_log.py)	100%	None planned
Current augmentation-strengthening experiment (in progress)	60%	Finish retrain (fold 3/5 as of this plan), run 4-way comparison, finalize eval_results/selected_models.json
User / developer manual	20% (README)	Expand into full user + developer manual

	only)	(Milestone 6)
--	-------	---------------

Milestone 4 (Sep 28): Itemized Tasks

1. Finish and evaluate the in-progress synthetic-augmentation experiment: complete the ResNet-50 and EfficientNet-B0 5-fold retrains, run the four-way threshold/ensembling comparison against the held-out test set, and update eval_results/selected_models.json per architecture based on the honestly measured outcome.
2. Implement overlay so the web app can show, alongside each model's verdict, a heatmap of which regions of the uploaded mammogram most influenced that verdict.
3. Polish and demo the two user-facing interfaces: add drag-and-drop upload, a loading indicator, and mobile-responsive styling to the Flask web app, and repair or remove the CompareModels.py dashboard's stale references to the nonexistent checkpoints_distill/plots_distill_kfold outputs.

Milestone 5 (Oct 26): Itemized Tasks

4. Implement, test, and demo the winning Milestone-4 modeling lever fully integrated into the live web app overlay rendered end-to-end on real uploaded images.
5. Conduct the full evaluation suite (accuracy/F1 on held-out data, latency measurement, reliability trial batch) and analyze the results against this semester's success criteria, recording honest outcomes in the project's specs/ ExecPlan history exactly as prior milestones have done.
6. Create the poster for the Senior Design Showcase, summarizing the system architecture, the honest measurement discipline that distinguishes this project, and the semester's headline result.

Milestone 6 (Nov 23): Itemized Tasks

7. Implement, test, and demo any remaining planned features (e.g., finalized UI polish, any additional explainability or reliability improvements identified during Milestone 5 evaluation).
8. Test and demo the entire system end-to-end, covering both the web application and the desktop comparison dashboard, on a fresh machine setup to confirm the system is reproducible outside the development environment.
9. Conduct final evaluation and analyze results, producing a complete before/after comparison across the full semester (baseline through Milestone 6) for both architectures.
10. Create the user/developer manual, expanding the current README into full setup, usage, and architecture-reference documentation.
11. Create the demo video showing the complete system in use, including the web app's upload-to-verdict flow and, if ready, the Grad-CAM explanation overlay.

Task Matrix for Milestone 4

Task	Kahlel Cardona	Woroma	Tara
1. Finish augmentation experiment retrain &	70%	30%	0%

4-way evaluation			
2. Implement & test app overlay	10%	30%	60%
3. Create web app	10%	30%	60%

Description of Each Planned Task for Milestone 4

Task 1: Complete the currently-running ResNet-50 and EfficientNet-B0 retrains under the strengthened data-augmentation configuration (see specs/synthetic-augmentation-f1-improvement.md), run the standard four-way threshold-tuning/ensembling comparison against the untouched 704-image held-out test set for each architecture, and update eval_results/selected_models.json only where the new result genuinely beats that architecture's original baseline, recording the honest outcome either way.

Task 2: Design and implement an overlay for both ResNet-50 and EfficientNet-B0, producing a heatmap image highlighting the regions of a mammogram that most influenced each model's verdict, and wire it into inference.py and the Flask web app so a user sees the heatmap alongside each model's label and confidence.

Task 3: Add drag-and-drop file upload, a visible loading indicator during model inference, and mobile-responsive CSS to the Flask web app's single page, and repair CompareModels.py's dashboard so it no longer references the nonexistent checkpoints_distill/ and plots_distill_kfold/ directories left over from an earlier, abandoned experiment.

Approval from Faculty Advisor

Signature: _____ Date: _____